



TEMA 10: Da Web ao Mobile — Estratégias de Apps

Compreenda o mapa das tecnologias modernas de desenvolvimento móvel e aprenda como converter sua aplicação Web (HTML, CSS e JavaScript) em um aplicativo instalável (PWA) e empacotável para Android (APK), inclusive **sem instalar Android Studio**, direto pelo GitHub.



Web Apps



PWA



Capacitor



GitHub Actions



React Native



Flutter



Kotlin



1. O Mapa Tecnológico: Como os Aplicativos São Criados?



A Pergunta Essencial: em vez de memorizar códigos de 4 ferramentas diferentes, o bom desenvolvedor aprende a responder:

"Qual é a melhor tecnologia para o problema, o prazo e a equipe que eu tenho hoje?"

O mercado de desenvolvimento de aplicativos organiza-se em **quatro grandes categorias** de acordo com a proximidade do hardware e a linguagem utilizada:

1. NATIVO PURO



Kotlin (Android) / Swift (iOS)

Linguagem oficial do sistema. Acesso 100% irrestrito aos sensores e máximo desempenho gráfico. *Desvantagem:* código separado para cada loja.

2. MULTIPLATAFORMA



React Native / Flutter

Gera apps para Android e iOS a partir de uma única base de código. React Native usa JavaScript/React; Flutter usa Dart.

3. PROGRESSIVE WEB APP



PWA (HTML + CSS + JS)

Um site moderno que pode ser **instalado** na tela inicial do celular pelo navegador, funcionando em tela cheia e offline.

4. EMPACOTADOR HÍBRIDO



Capacitor / Cordova

Pega a aplicação Web existente e a embrulha dentro de um container nativo (WebView), gerando um arquivo `.apk`.



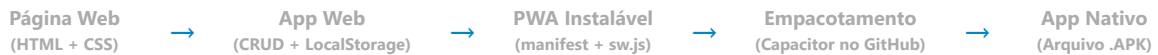
Comparativo Direto das Abordagens

Tecnologia	Linguagem Base	Plataformas	Curva para quem sabe JS	Cenário Ideal
PWA	HTML / CSS / JS	Android, iOS, PC, Mac	★★★★★ (Imediata)	Sistemas internos e CRUDs sem loja.
Capacitor	HTML / CSS / JS	Android e iOS (APK/IPA)	★★★★★ (Muito rápida)	Levar um sistema Web existente para as lojas.
React Native	JavaScript / React	Android e iOS	★★★★ (Média)	Apps comerciais com equipe Web.
Flutter	Dart	Android, iOS, Web, Desktop	★★★ (Nova sintaxe)	Apps com animações refinadas.
Kotlin	Kotlin	Android puro	★ (Ecossistema JVM)	Controle fino de hardware.



2. A Trilha Estratégica: Da Web ao APK

Se a turma já domina HTML, CSS, JS e LocalStorage, o caminho mais produtivo não é começar do zero com nova linguagem, mas evoluir em degraus:



✂ O Que Torna um Site uma PWA?

Apenas 3 requisitos fundamentais:

1. **Design Responsivo:** a tela se adapta ao tamanho dos smartphones.
2. **Manifesto (`manifest.json`):** nome, ícones, cor da barra de status.
3. **Service Worker (`sw.js`):** cache offline e instalação segura.
4. **HTTPS:** exigência do navegador para habilitar a instalação.

🔧 3. Laboratório Prático: Transformando o CRUD em App

Acompanhe os **7 passos reais** para transformar o CRUD de Alunos em app instalável (PWA) e empacotável (APK), **100% gratuito pelo GitHub**. Use o botão **Copiar** de cada passo para levar o código direto ao seu repositório.

[1. index.html](#)[2. style.css](#)[3. app.js](#)[4. manifest.json](#)[5. sw.js](#)[6. build.yml](#)[7. Publicar](#)

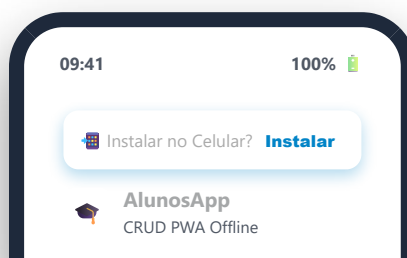
Passo 1: `www/index.html`

Página principal do app. Note o `<meta viewport>` (adapta ao celular), o `<link rel="manifest">` (habilita a PWA) e o `<link rel="icon">` .

`www/index.html`

Copiar

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
  <title>AlunosApp</title>
  <link rel="stylesheet" href="style.css">
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#0284c7">
  <link rel="icon" type="image/png" sizes="192x192" href="icons/icon-192.png">
</head>
<body>
  <div class="container">
    <header class="topo">
      <div class="icone">🎓</div>
      <div>
        <h1>AlunosApp</h1>
        <p>Gestão Escolar</p>
      </div>
    </header>
  </div>
</body>
</html>
```



Nome Completo

E-mail

Idade

Salvar Aluno

ID	NOME	IDADE	
#1	Carlos Silva	22	X
#2	Ana Souza	19	X

4. Gerando o APK sem Android Studio: GitHub Actions

💡 **Novidade importante:** hoje é possível gerar o APK **direto pelo navegador**, sem instalar Node.js, Android Studio nem configurar SDK local. O **GitHub Actions** é um serviço gratuito que roda os comandos do Capacitor em servidores na nuvem e devolve o APK pronto para download.

Estrutura do Repositório

```

seu-repo/
├── .github/
│   └── workflows/
│       └── build.yml      ← gera o APK na nuvem
└── www/
    ├── index.html
    ├── style.css
    ├── app.js
    ├── manifest.json
    ├── sw.js
    └── icons/
        ├── icon-192.png
        └── icon-512.png
  
```

O que o workflow faz

1. **A cada push em main** : publica **www/** no GitHub Pages (link HTTPS público, com PWA instalável).
2. **A cada tag v*** : instala Capacitor, compila o APK e anexa numa *Release* do GitHub.
3. Você baixa o APK por github.com/SEU-USUARIO/SEU-REPO/releases/download/v1.0.0/app-debug.apk — link HTTPS que o Chrome não bloqueia.

✅ **Zero instalação local.** Todo o build acontece nos servidores do GitHub. O aluno só precisa do navegador.

⚠️ **Requisito:** antes de rodar, é obrigatório ativar em *Settings* → *Pages* → *Source*: **GitHub Actions**. Sem isso, o job **deploy-pwa** falha com **Get Pages site failed**.

5. Alternativa: Capacitor Instalado Localmente

💡 **Quando usar:** se a rede da escola permitir instalar Node.js e Android Studio no computador, o processo local também funciona. É o mesmo que o GitHub Actions faz nos bastidores, mas rodando na sua máquina.

Para o desenvolvedor Web, o fluxo de geração de um APK local torna-se:

```
// 1. Inicializa o Capacitor no projeto existente
npm install @capacitor/core @capacitor/cli
npx cap init "AlunosApp" "com.escola.alunosapp" --web-dir www

// 2. Adiciona a plataforma Android
npm install @capacitor/android
npx cap add android

// 3. Copia o HTML/CSS/JS e abre no Android Studio
npx cap copy
npx cap open android
```

💡 **Vantagem estrutural:** o **LocalStorage**, o **CRUD** e toda a lógica JavaScript continuam funcionando idênticos no celular, mas agora com ícone na gaveta de aplicativos.

📄 6. Questionário de Fixação (17 Questões)

Avalie seu domínio sobre a arquitetura de aplicativos modernos, PWA, Capacitor e GitHub Actions:

1 **Qual é a principal linguagem utilizada para desenvolver aplicativos nativos para o sistema operacional Android oficial da Google?**

☐ PHP

☒ Kotlin

☐ Ruby

☐ CSS Puro

2 **O framework React Native permite criar aplicativos para Android e iOS utilizando qual base de programação?**

☐ C++ e Assembly

☒ JavaScript e a biblioteca React

☐ Python e Django

☐ Apenas comandos SQL

3 **O framework Flutter, criado pela Google, utiliza nativamente qual linguagem de programação?**

☒ Dart

☐ Java 8

☐ TypeScript☐ Swift

4 O que significa a sigla PWA no contexto de desenvolvimento de software?

☐ Personal Web Assistant☒ Progressive Web Apps (Aplicativos Web Progressivos)☐ Programming With Android☐ Public Wireless Access

5 Qual é o arquivo responsável por informar ao sistema operacional o nome do app, os ícones de inicialização e as cores da barra em uma PWA?

☐ style.css☒ manifest.json☐ database.sql☐ package-lock.xml

6 Se você já possui um sistema CRUD em HTML, CSS e JavaScript, qual é o caminho mais rápido para transformá-lo em aplicativo sem reaprender outra linguagem?

☐ Reescrever tudo do zero em C#☒ Configurar como PWA ou empacotar via Capacitor☐ Apenas mudar a extensão do arquivo para .exe☐ Instalar uma placa de vídeo nova

7 No `manifest.json`, o que a propriedade `"display": "standalone"` faz ao abrir o app instalado?

☒ Abre em tela cheia, ocultando a barra de navegação do browser☐ Bloqueia o toque na tela☐ Exclui o LocalStorage☐ Obriga o uso de teclado físico

8 Qual é o papel da ferramenta Capacitor no ecossistema mobile moderno?

☐ Criar bancos de dados relacionais na nuvem

☒ Empacotar arquivos Web (HTML/JS/CSS) em um app binário nativo (APK) através de uma WebView

☐ Substituir o processador do celular

☐ Converter imagens PNG em arquivos de texto

9 Qual meta tag é obrigatória no `<head>` para que o layout se ajuste à largura das telas dos celulares?

☐ `<meta charset="UTF-8">`

☒ `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

☐ `<meta http-equiv="refresh">`

☐ `<meta name="author">`

10 Por que uma PWA consegue continuar exibindo dados mesmo sem conexão com a internet?

☐ Porque ela gera sinal 5G próprio

☒ Porque utiliza recursos locais, como o LocalStorage gravado no disco do dispositivo

☐ Porque o cabo de energia mantém os dados na memória

☐ PWAs não funcionam offline de forma alguma

11 Qual é a principal vantagem de escolher uma abordagem multiplataforma (React Native ou Flutter) em vez de nativa pura?

☒ Manter uma única base de código para atender Android e iOS

☐ Garantir que o app nunca terá bugs

☐ Não precisar publicar na Google Play Store

☐ Fazer o download do app ser instantâneo sem internet

12 Ao adicionar o campo "Idade" no formulário do CRUD Web e empacotá-lo como app móvel, o que acontece com a lógica JavaScript?

☐ Ela para de funcionar porque celular não roda JavaScript

☒ Ela continua funcionando, pois o WebView executa o motor JavaScript normalmente

☐ Exige converter todos os números para texto

☐ O botão salvar é desativado permanentemente

13 Para instalar uma PWA no celular, o usuário é obrigado a pagar a taxa de desenvolvedor da Google Play.

☒ Verdadeiro

☐ Falso

14 O Kotlin possui interoperabilidade total com Java, permitindo que bibliotecas clássicas rodem em projetos modernos de Kotlin.

☒ Verdadeiro

☐ Falso

15 O `manifest.json` é vinculado ao HTML através de `<link rel="manifest" href="manifest.json">` .

☒ Verdadeiro

☐ Falso

16 É possível gerar o APK de um aplicativo Web sem instalar o Android Studio no computador, utilizando apenas o GitHub Actions.

☒ Verdadeiro

☐ Falso

17 O `sw.js` (Service Worker) é o arquivo responsável por permitir que a PWA funcione offline, guardando arquivos em cache.

☒ Verdadeiro

☐ Falso

☒ Verificar Respostas

☐ Limpar Respostas

Resultado da Avaliação

17 / 17

Aproveitamento: 100%